



MANIPAL

ACADEMY *of* HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

CYE 5151 Cryptology Lab – 3

Reg. Number	Name	Branch
261100690010	Hegde Aansh Dilip	Cyber Security

04/09/2026



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Part A: Python Basics and Debugging

1. Debug the following programs. Identify the type of error in each case.

```
print("Welcome to Cryptology Lab)

message = "Python Programming"

print(mesage)

num = input("Enter a number: ")

result = num + 10

print(result)
```

Line	Error	Type of Error	Correction
1	Missing closing "	Syntax Error	print("Welcome to Cryptology Lab")
3	mesage is misspelled	Name Error	print(message)
5	input() returns a string, but 10 is an integer	Type Error	Convert input using int()

2. Write Python statements to display the following exactly:

Welcome to Cryptology Lab

It's interesting to learn Python.

Students, "Welcome to the Cryptology Lab".

Answer:

```
print("Welcome to Cryptology Lab")

print("It's interesting to learn Python.")

print('Students, "Welcome to the Cryptology Lab".')
```

3. Read two numbers from the user and perform:

Addition, Subtraction, Multiplication, Division, Integer division, Remainder, Exponentiation

Display each result with an appropriate message.

```
1 num1 = int(input("Enter first number: "))
2 num2 = int(input("Enter second number: "))
3
4 print("Addition:", num1 + num2)
5 print("Subtraction:", num1 - num2)
6 print("Multiplication:", num1 * num2)
7 print("Division:", num1 / num2)
8 print("Integer Division:", num1 // num2)
9 print("Remainder:", num1 % num2)
10 print("Exponentiation:", num1 ** num2)
```

```
Enter first number: 20
Enter second number: 10
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2.0
Integer Division: 2
Remainder: 0
Exponentiation: 10240000000000

=== Code Execution Successful ===
```

4. Read a floating-point number and display:

its data type, its absolute value; the value rounded to two decimal places.

```
1 num = float(input("Enter a floating-point number: "))
2
3 print("Data type:", type(num))
4 print("Absolute value:", abs(num))
5 print("Rounded value:", round(num, 2))
```

Enter a floating-point number: 314
Data type: <class 'float'>
Absolute value: 314.0
Rounded value: 314.0

=== Code Execution Successful ===

Part B — Data Types and Collections (~25 min)

5. Create variables containing examples of:

int, float, complex, str, list, tuple, set, and dictionary.

Use Python to display the value and data type of each variable.

```
1 a = 10
2 b = 10.5
3 c = 2 + 3j
4 d = "Python"
5 e = [1, 2, 3]
6 f = (4, 5, 6)
7 g = {7, 8, 9}
8 h = {"name": "John", "age": 20}
9
10 print(a, type(a))
11 print(b, type(b))
12 print(c, type(c))
13 print(d, type(d))
14 print(e, type(e))
15 print(f, type(f))
16 print(g, type(g))
17 print(h, type(h))
```

10 <class 'int'>
10.5 <class 'float'>
(2+3j) <class 'complex'>
Python <class 'str'>
[1, 2, 3] <class 'list'>
(4, 5, 6) <class 'tuple'>
{8, 9, 7} <class 'set'>
{'name': 'John', 'age': 20} <class 'dict'>

=== Code Execution Successful ===

6. Demonstrate experimentally that:

Strings are immutable, whereas lists are mutable.

Include the Python statements you used and explain the observed result.

```
1 text = "Hello"
2 print("String:", text)
3
4 text[0] = "J"
```

String: Hello
ERROR!
Traceback (most recent call last):
 File "<main.py>", line 4, in <module>
TypeError: 'str' object does not support item assignment

=== Code Exited With Errors ===

```
1 my_list = ["H", "e", "l", "l", "o"]
2 print("Original list:", my_list)
3 my_list[0] = "J"
4 print("Modified list:", my_list)
```

Original list: ['H', 'e', 'l', 'l', 'o']
Modified list: ['J', 'e', 'l', 'l', 'o']

=== Code Execution Successful ===

7. Create a list containing five prime numbers. Perform the following:

display the first three elements; display the last element; append another prime number; extend the list with two more prime numbers; display the total number of elements.

```
1 prime = [2, 3, 5, 7, 11]
2 print("First three elements:", prime[:3])
3 print("Last element:", prime[-1])
4 prime.append(13)
5 print("After adding one prime:", prime)
6 prime.extend([17, 19])
7 print("After adding two more primes:", prime)
8 print("Total number of elements:", len(prime))
```

First three elements: [2, 3, 5]
Last element: 11
After adding one prime: [2, 3, 5, 7, 11, 13]
After adding two more primes: [2, 3, 5, 7, 11, 13, 17, 19]
Total number of elements: 8

=== Code Execution Successful ===

Part C — String Operations

For Questions 8–12, use:

```
message = "CRYPTOLOGY USING PYTHON"
```

8. Display:

length of the string; first five characters; last six characters; characters from positions 3 to 8; string in reverse order.

```
1 message = "CRYPTOLOGY USING PYTHON"
2 print("Length:", len(message))
3 print("First five characters:", message[:5])
4 print("Last six characters:", message[-6:])
5 print("Characters from position 3 to 8:", message[3:9])
6 print("Reverse:", message[::-1])
```

Length: 23
First five characters: CRYPT
Last six characters: PYTHON
Characters from position 3 to 8: PTOLG
Reverse: NOHTYP GNISU YGOLOTPYRC

=== Code Execution Successful ===

9. Perform the following:

convert to lowercase; convert back to uppercase; check whether "PYTHON" exists in the string; replace "PYTHON" with "PROGRAMMING".

```
message = "CRYPTOLOGY USING PYTHON"

print("Lowercase:", message.lower())
print("Uppercase:", message.upper())

print("PYTHON exists:", "PYTHON" in message)

new_message = message.replace("PYTHON", "PROGRAMMING")
print("After replacement:", new_message)
```

Lowercase: cryptology using python
Uppercase: CRYPTOLOGY USING PYTHON
PYTHON exists: True
After replacement: CRYPTOLOGY USING PROGRAMMING

=== Code Execution Successful ===

10. Remove the spaces from the string and display the resulting string.

```
1 message = "CRYPTOLOGY USING PYTHON"
2
3 message = message.replace(" ", "")
4
5 print("String without spaces:", message)
```

String without spaces: CRYPTOLOGYUSINGPYTHON

=== Code Execution Successful ===

11. Using a loop, display every character and its corresponding Unicode/ASCII value using ord().

Example:

C -> 67

R -> 82

Y -> 89

...

```

message = "CRYPTOLOGY USING PYTHON"

for ch in message:
    print(ch, "->", ord(ch))

```

```

C -> 67
R -> 82
Y -> 89
P -> 80
T -> 84
O -> 79
L -> 76
O -> 79
G -> 71
Y -> 89
- -> 32
U -> 85
S -> 83
I -> 73
N -> 78
G -> 71
- -> 32
P -> 80
Y -> 89
T -> 84
H -> 72
O -> 79
N -> 78

```

=== Code Execution Successful ===

12. Use chr() to convert the following values into characters:

65, 67, 73, 80, 72, 69, 82

Combine the resulting characters and identify the word.

```

values = [65, 67, 73, 80, 72, 69, 82]
word = ""
for value in values:
    word = word + chr(value)
print("Characters:", word)
print("Word:", word)

```

```

Characters: ACIPHER
Word: ACIPHER

```

=== Code Execution Successful ===

Part D — Text Processing for Cryptology

13. Plaintext preprocessing

Read a message from the user and: remove leading/trailing spaces; convert it to uppercase; remove spaces between words.

Example:

Input : meet me tomorrow

Output: MEETMETOMORROW

```

1 # Plaintext Preprocessing
2
3 message = input("Enter a message: ")
4
5 # Remove leading and trailing spaces
6 message = message.strip()
7
8 # Convert to uppercase
9 message = message.upper()
10
11 # Remove spaces between words
12 message = message.replace(" ", "")
13
14 print("Output:", message)

```

```

Enter a message: meet me tomorrow
Output: MEETMETOMORROW

```

=== Code Execution Successful ===

14. Character grouping: Read a string and divide it into blocks of five characters.

Example: Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LAB

```
1 message = input("Enter a string:")
2 print ("Output:")
3
4 for i in range(0,len(message),5):
5     print(message[i:i+5])
```

Enter a string:CRYPTOLOGYLAB
Output:
CRYPT
LOGY
LAB
=== Code Execution Successful ===

15. Padding

Modify Question 14 so that an incomplete final block is padded with X.

Input: CRYPTOLOGYLAB

Output:

CRYPT

LOGY

LABXX

```
1 message = input("Enter a string: ")
2
3 print("Output:")
4
5 for i in range(0, len(message), 5):
6     block = message[i:i+5]
7
8     if len(block) < 5:
9         block = block + "X" * (5 - len(block))
10
11     print(block)
```

Enter a string: CRYPTOLOGYLAB
Output:
CRYPT
LOGY
LABXX
=== Code Execution Successful ===

16. Character frequency

For the following text:

text = "CRYPTOLOGYISINTERESTING"

write a program to calculate the frequency of each character.

Then identify the most frequently occurring character.

```
1 text = input
2
3 frequency = {}
4
5 for ch in text:
6     if ch in frequency:
7         frequency[ch] += 1
8     else:
9         frequency[ch] = 1
10
11 print("Character frequency:")
12
13 for ch, count in frequency.items():
14     print(ch, ":", count)
15
16 most_frequent = max(frequency, key=frequency.get)
17
18 print("Most frequently occurring character:", most_frequent)
19 print("Frequency:", frequency[most_frequent])
```

Character frequency:
C : 1
R : 2
Y : 2
P : 1
T : 3
O : 2
L : 1
G : 2
I : 3
S : 2
N : 2
E : 2
Most frequently occurring character: T
Frequency: 3

=== Code Execution Successful ===

17. Percentage frequency

Extend Question 16 to calculate:

Percentage Frequency = Total Number of Characters/ Frequency of Character ×100

Display the percentage rounded to two decimal places.

```
1 text = "CRYPTOLOGYISINTERESTING"
2
3 frequency = {}
4
5 for ch in text:
6     if ch in frequency:
7         frequency[ch] += 1
8     else:
9         frequency[ch] = 1
10
11 total = len(text)
12
13 print("Character Frequency and Percentage:")
14
15 for ch, count in frequency.items():
16     percentage = (count / total) * 100
17     print(ch, ":", count, "->", round(percentage, 2), "%")
```

Character Frequency and Percentage:
C : 1 -> 4.35 %
R : 2 -> 8.7 %
Y : 2 -> 8.7 %
P : 1 -> 4.35 %
T : 3 -> 13.04 %
O : 2 -> 8.7 %
L : 1 -> 4.35 %
G : 2 -> 8.7 %
I : 3 -> 13.04 %
S : 2 -> 8.7 %
N : 2 -> 8.7 %
E : 2 -> 8.7 %

=== Code Execution Successful ===

Part E — Mathematical Foundations for Cryptology

18. Modular arithmetic

Use Python to evaluate: $29 \bmod 26$, $55 \bmod 26$, $78 \bmod 26$, $-3 \bmod 26$, $-29 \bmod 26$

Then write a program that accepts an integer and calculates the integer modulo 26.

```
1 print("29 mod 26 =", 29 % 26)
2 print("55 mod 26 =", 55 % 26)
3 print("78 mod 26 =", 78 % 26)
4 print("-3 mod 26 =", -3 % 26)
5 print("-29 mod 26 =", -29 % 26)
6
7 num = int(input("Enter an integer: "))
8
9 result = num % 26
10
11 print("Integer modulo 26 =", result)
```

```
29 mod 26 = 3
55 mod 26 = 3
78 mod 26 = 0
-3 mod 26 = 23
-29 mod 26 = 23
Enter an integer: 51
Integer modulo 26 = 25

=== Code Execution Successful ===
```

19. Alphabet representation

Using A=0, B=1,...,Z=25

write a program that accepts an uppercase character and displays its numerical representation.

Example:

Enter character: M

M -> 12

Also perform the reverse conversion:

Enter number: 12

12 -> M

```
1 ch = input("Enter character: ")
2
3 number = ord(ch) - ord('A')
4
5 print(ch, "->", number)
6
7 num = int(input("Enter number: "))
8
9 character = chr(num + ord('A'))
10
11 print(num, "->", character)
```

```
Enter character: M
M -> 12
Enter number: 12
12 -> M

=== Code Execution Successful ===
```


20. Prime numbers Write a program to:

determine whether a given number is prime;

display all prime numbers within a range specified by the user.

```
1 num = int(input("Enter a number: "))
2
3 if num < 2:
4     print(num, "is not a prime number")
5 else:
6     prime = True
7
8     for i in range(2, num):
9         if num % i == 0:
10             prime = False
11             break
12
13 if prime:
14     print(num, "is a prime number")
15 else:
16     print(num, "is not a prime number")
17
18 start = int(input("Enter starting number: "))
19 end = int(input("Enter ending number: "))
20
21 print("Prime numbers in the range:")
22
23 for num in range(start, end + 1):
24     if num < 2:
25         continue
26
27     prime = True
28
29     for i in range(2, num):
30         if num % i == 0:
31             prime = False
32             break
33
34     if prime:
35         print(num, end=" ")
36
37 Enter a number: 17
38 17 is a prime number
39 Enter starting number: 19
40 Enter ending number: 30
41 Prime numbers in the range:
42 19 23 29
43 === Code Execution Successful ===
```

21. GCD and coprimes

Read two integers from the user. Find their GCD and determine whether they are coprime.

```
1 import math
2
3 a = int(input("Enter first number: "))
4 b = int(input("Enter second number: "))
5
6 gcd = math.gcd(a, b)
7
8 print("GCD =", gcd)
9
10 if gcd == 1:
11     print("The numbers are coprime.")
12 else:
13     print("The numbers are not coprime.")
14
15 Enter a number: 17
16 17 is a prime number
17 Enter starting number: 19
18 Enter ending number: 30
19 Prime numbers in the range:
20 19 23 29
21 === Code Execution Successful ===
```

22. Write a program to get the multiplicative inverse of an integer b.

```
1 b = int(input("Enter an integer: "))
2
3 inverse = -1
4
5 for x in range(1, 26):
6     if (b * x) % 26 == 1:
7         inverse = x
8         break
9
10 if inverse == -1:
11     print("Multiplicative inverse does not exist.")
12 else:
13     print("Multiplicative inverse of", b, "mod 26 is:", inverse)
```

Enter an integer: 5
Multiplicative inverse of 5 mod 26 is: 21
=== Code Execution Successful ===

Part F — Challenge / If Time Permits (~15 min)

23. WAP in python to implement client and server communication.

```
GNU nano 9.0 server.py
import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(("127.0.0.1", 12345))
server_socket.listen(1)
print("Server is waiting for connection...")
client_socket, address = server_socket.accept()
print("Client connected:", address)
message = client_socket.recv(1024).decode()
print("Message from client:", message)
client_socket.send("Hello from Server".encode())
client_socket.close()
server_socket.close()
```

```
GNU nano 9.0 client.py
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 12345))
message = input("Enter message: ")
client_socket.send(message.encode())
response = client_socket.recv(1024).decode()
print("Message from server:", response)
client_socket.close()
```

```
(kali㉿kali)-[~]
└─$ python server.py
Server is waiting for connection...
Client connected: ('127.0.0.1', 57438)
Message from client: Hello Aansh
```

```
(kali㉿kali)-[~]
└─$ python client.py
Enter message: Hello Aansh
Message from server: Hello from Server
```